CATENA DI SUPERMERCATI

TRACCIA:

Il proprietario di una catena di supermercati intende creare una tabella per una più efficiente gestione dei suoi supermercati. Per ogni supermercato il sistema deve memorizzare il nome e il cognome e il responsabile del supermercato, l'indirizzo del supermercato, il numero di dipendenti, il fatturato dell'ultimo anno.
Il programmatore dovrà consentire di:

- Caricare le informazioni della tabella
- visualizzare i dati del supermercato il cui responsabile è inserito in input dal proprietario della catena
- visualizzare quanti supermercati sono presenti nella città inserita in input dal proprietario della catena
- visualizzare il supermercato che ha il maggior numero di dipendenti

1. Analizzare il problema chiarendo le sue specifiche e formando le eventuali ipotesi. Individuare i dati di ingresso e fornire un campione. Individuare i dati di uscita o risultati e fornire il campione associato al campione di ingresso.
2. Progettare la strategia di soluzione individuando una scomposizione del problema in sottoproblemi e rappresentarla mediante uno schema (ex. l'albero di sviluppo)
3. Descrivere la strategia di soluzione mediante un linguaggio di descrizione.

ANALISI

CHIARIFICA:

Creare un array di record in cui si memorizzano i dati di alcuni supermercati, i quali dati sono:
il nome e cognome del responsabile, la città del negozio, la via di dove esso è situato, il numero di dipendenti presenti nella struttura e il fatturato annuo di quest'ultima.
Inoltre il programma dovrà avere delle funzioni e/o procedure che permettano ,in questo preciso ordine, il caricamento dei dati dei supermercati nell'array; ricercare i dati del supermercato filtrandoli per nome e cognome del responsabile inseriti dall'utente, in caso non sia presente nessun risultato viene comunicato un messaggio di errore; visualizzare in output il numero dei supermercati presenti in una determinata città definita in input dall'utente (attraverso l'utilizzo di un contatore), anche in questo contesto se non è presente nessun risultato viene comunicato un messaggio di errore ed infine cercare il supermercato con più dipendenti tra tutti i supermercati presenti nell'array di record e visualizzare a schermo i dati del supermercato che ha il maggior numero di dipendenti, confrontandoli tra di loro.

Si ipotizza che ogni supermercato sia gestito da un singolo responsabile, che siano presenti sempre dei dipendenti e che i supermercati possano trovarsi in città diverse e anche più supermercati nella medesima città.
Inoltre l'utente sarà costretto ad inserire almeno 1 supermercato.

DATI:

Dati di Input	Variabili di Input
Numero supermercati	`num_sup` (intero > 0)
Per ogni supermercato:	`Sup` (array di record)
- Cognome del responsabile	`Sup[i].Cognome_resp` (stringa `char`)
- Nome del responsabile	`Sup[i].nome_resp` (stringa `char`)
- Via	`Sup[i].via` (stringa `char`)
- Città	`Sup[i].citta` (stringa `char`)
- Numero di dipendenti	`Sup[i].num_dip` (intero > 0)
- Fatturato dell'anno	`Sup[i].fatt` (float)
Cognome responsabile ricercato	`Cogn_ricerca` (stringa `char`)
Nome responsabile ricercato	`Nome_ricerca` (stringa `char`)
Città ricercata	`Citta_ricerca` (stringa `char`)

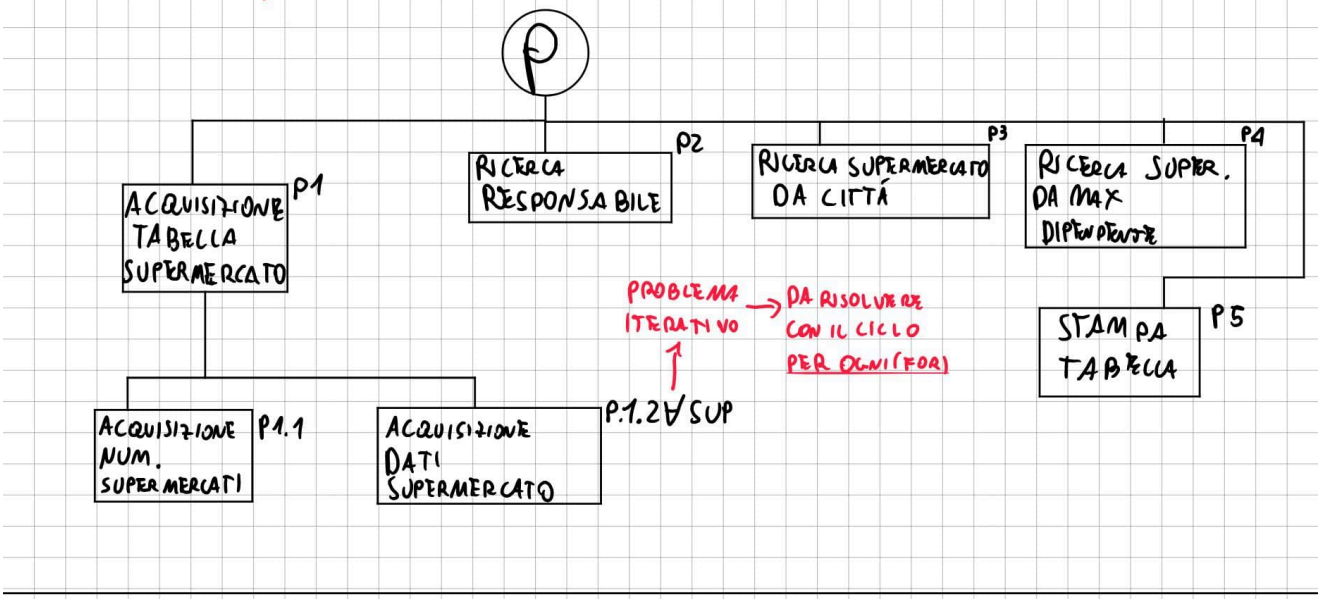
Dati di Output	Variabili di Output
Dati del supermercato il cui responsabile è il nominativo inserito in input	`Sup[i].Cognome_resp`, `Sup[i].nome_resp`, `Sup[i].via`, `Sup[i].citta`, `Sup[i].num_dip`, `Sup[i].fatt`
Oppure messaggio di inesistenza del responsabile inserito	`"Il nominativo inserito non è responsabile di alcun supermercato"`
Numero supermercati presenti nella città inserita in input	`Totale_supermercati` (intero >= 0)
Oppure messaggio di inesistenza supermercati nella città inserita in input	`"Nella città inserita non esistono supermercati"`
Dati del supermercato con numero massimo di dipendenti	`Sup[max].Cognome_resp`, `Sup[max].nome_resp`, `Sup[max].via`, `Sup[max].citta`, `Sup[max].num_dip`, `Sup[max].fatt`

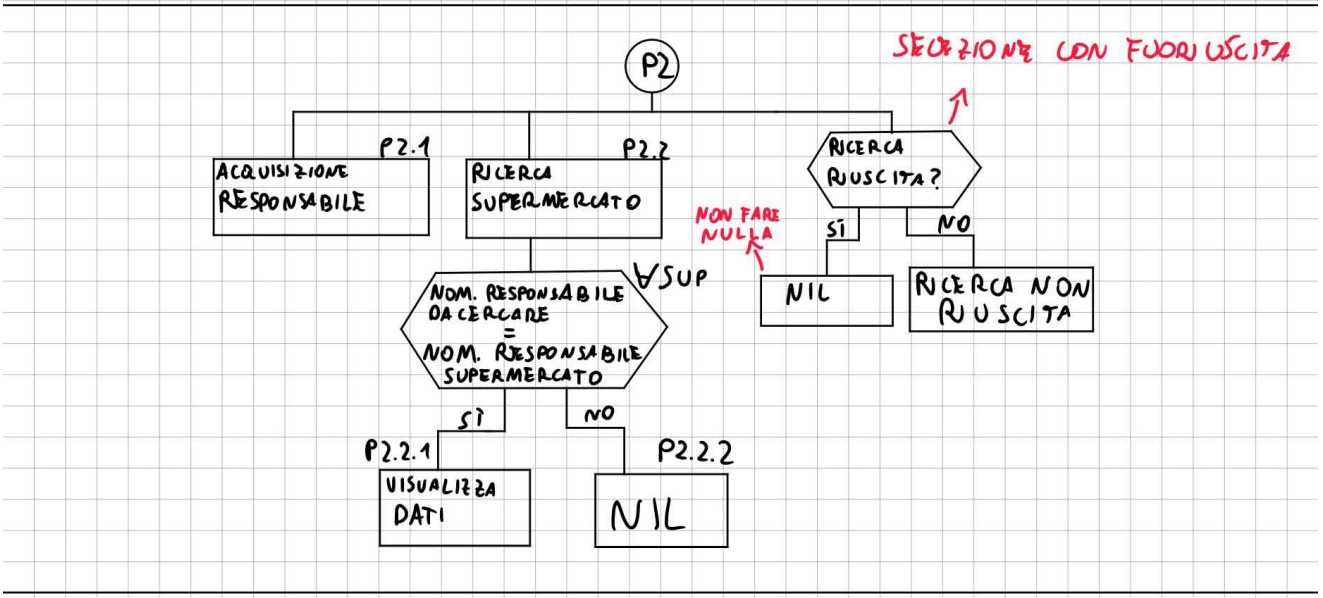
Nome variabile	Tipo	Descrizione	Ambito
`STRINGA`	`#define`	Costante per definire la lunghezza massima delle stringhe.	Globale
`MAX_SUP`	`#define`	Costante per definire il numero massimo di supermercati gestibili.	Globale
`supermercato_t`	`struct`	Struttura che contiene i dati di un supermercato (cognome, nome, città, num_dip, fattura).	Globale
`num_sup`	`int`	Numero di supermercati specificato dall'utente.	`caricamento`, `conta_supermercati`
`totale_supermercati`	`int`	Numero totale di supermercati trovati nella città specificata.	`conta_supermercati`
`massimo_idx`	`int`	Indice del supermercato con il numero massimo di dipendenti.	`massimo_dipendenti`
`flag`	`short`	Indicatore booleano per verificare se un responsabile è stato trovato nella ricerca.	`ricerca_responsabile`

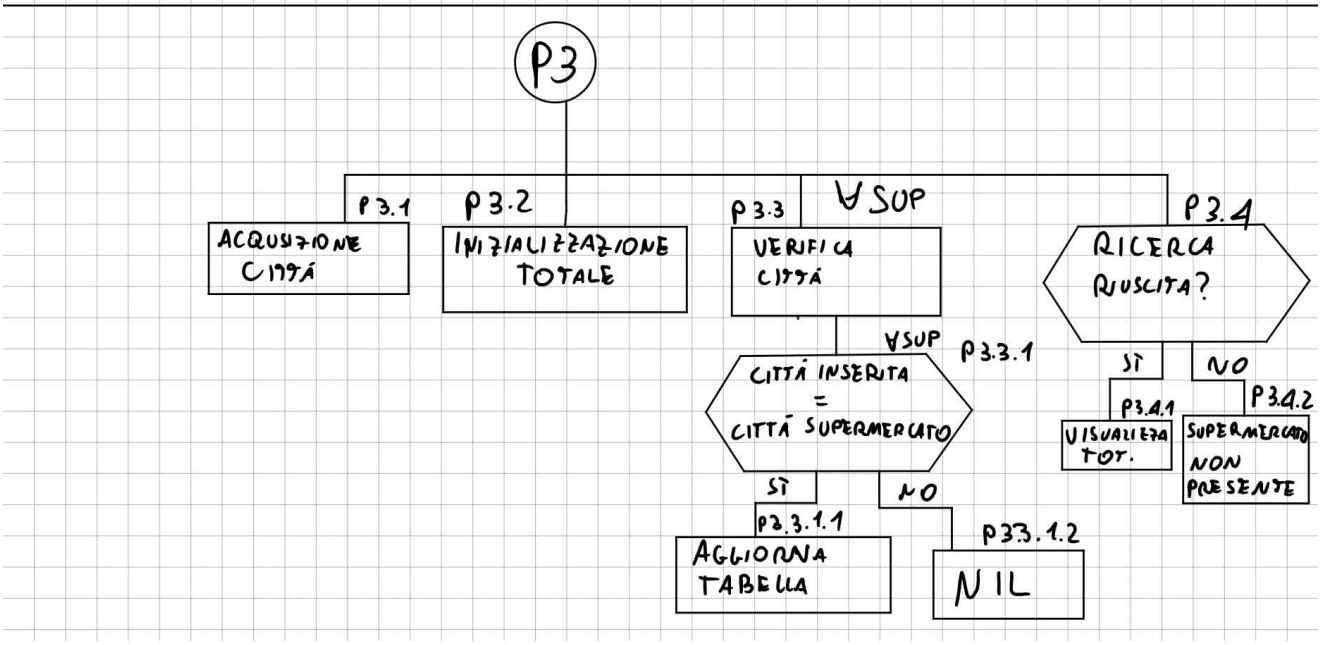
SCOMPOSIZIONE IN SOTTOPROBLEMI ATTRAVERSO METODO GRAFICO

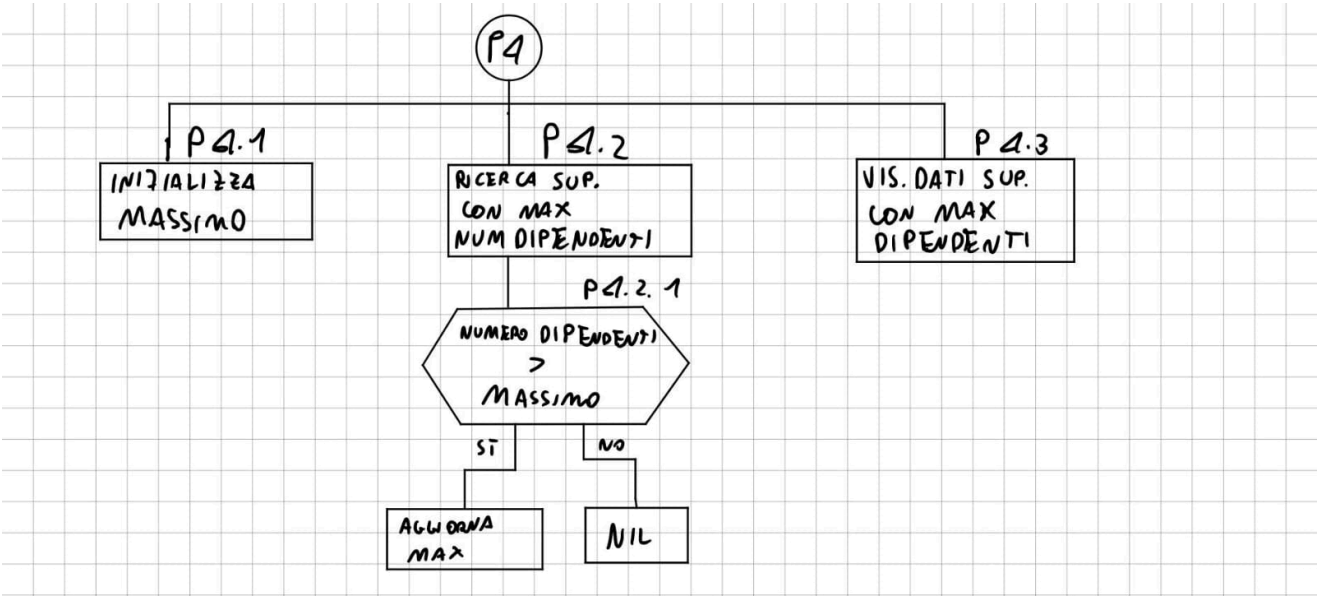
ALBERO DI DECOMPOSIZIONE / SVILUPPO:

SUPERMERCATO - ALBERO DI DECOMPOSIZIONE



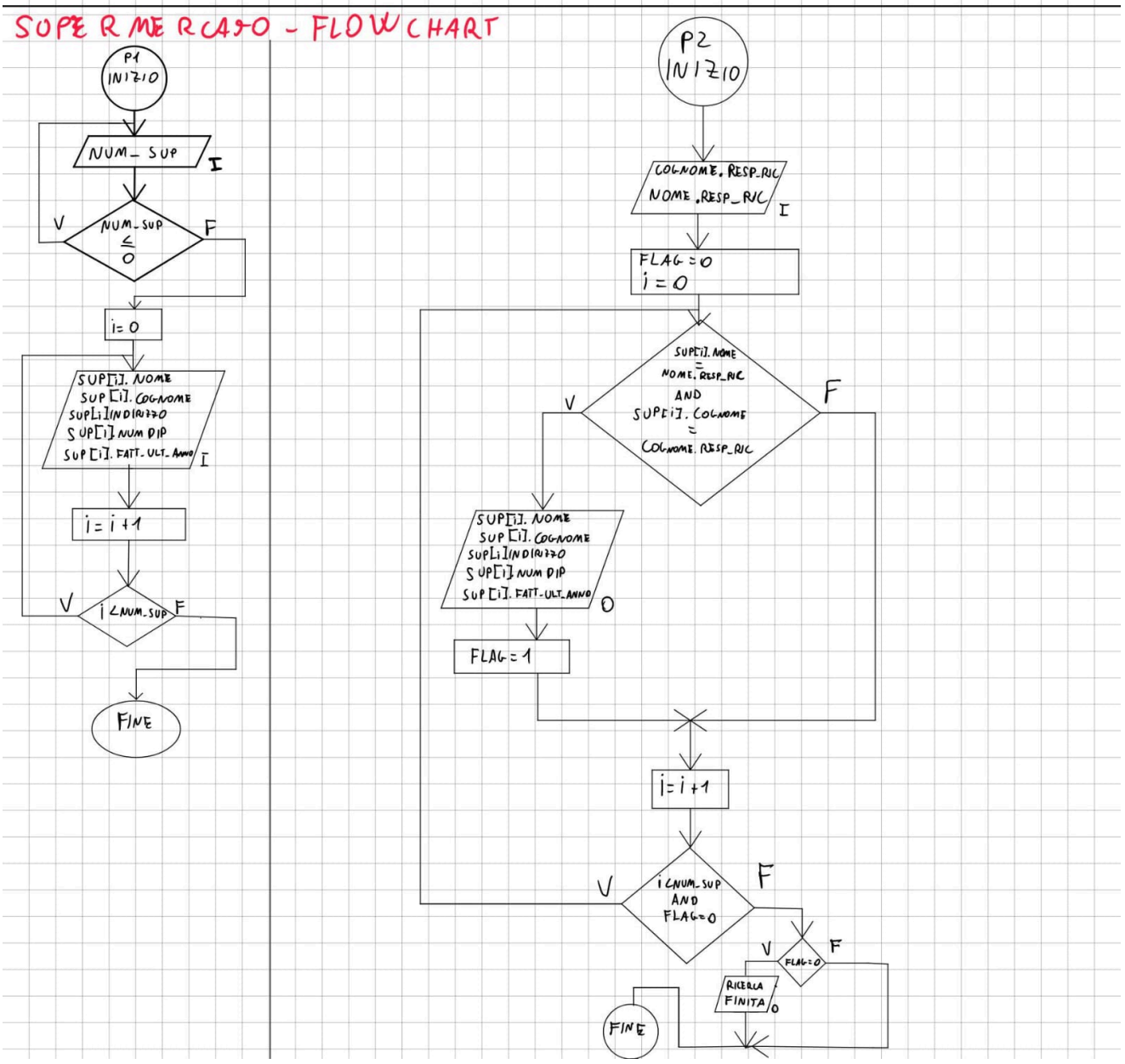


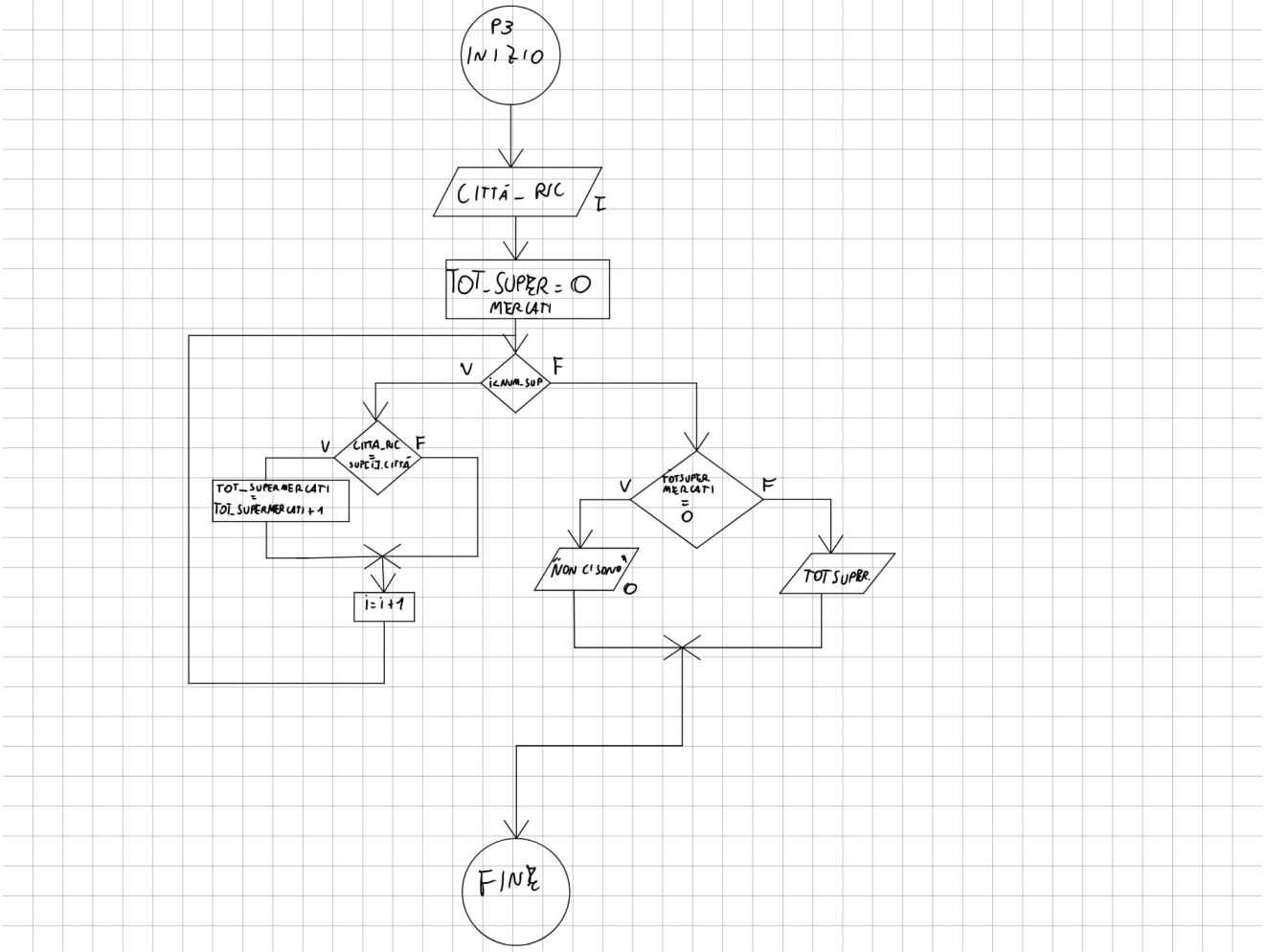


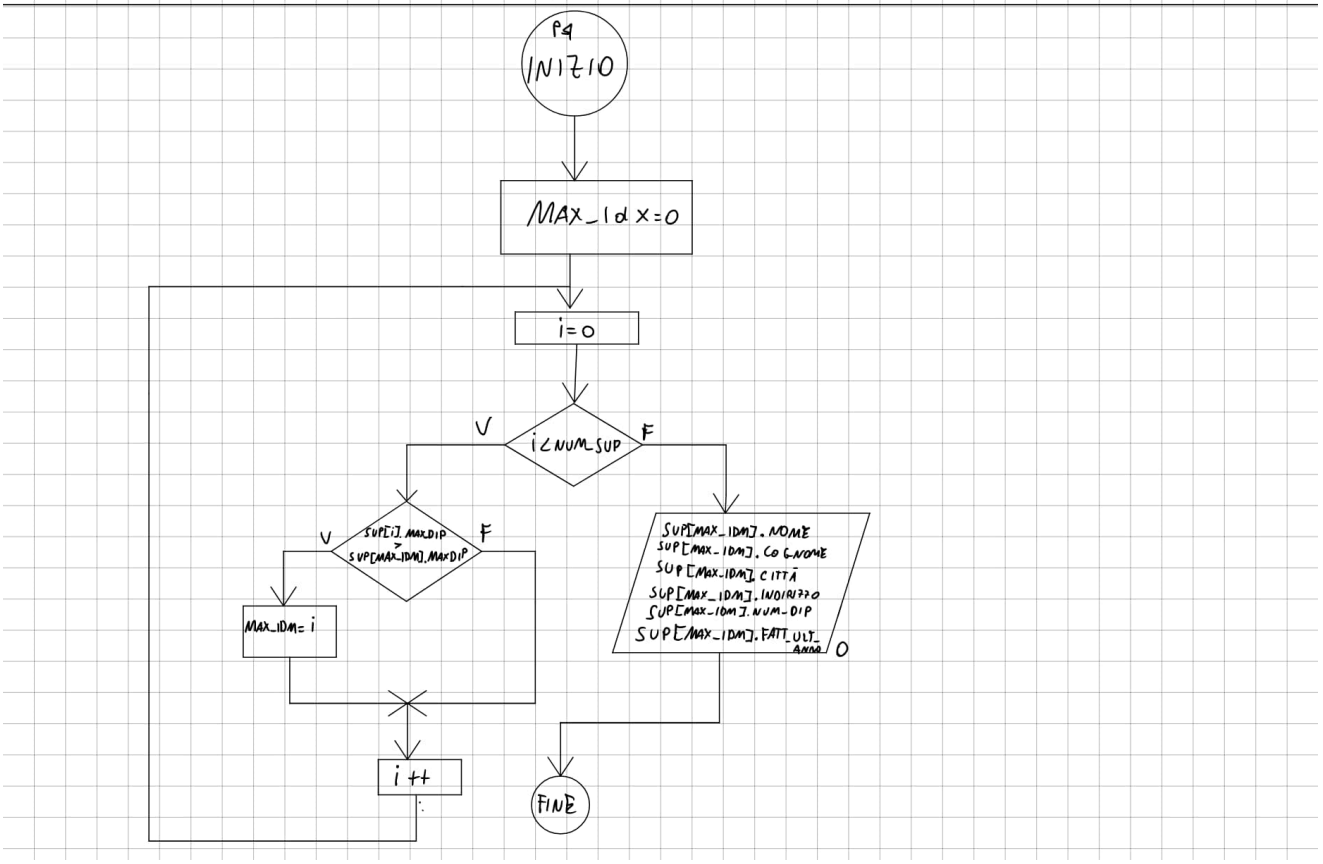


RAPPRESENTAZIONE GRAFICA DELLA SOLUZIONE

FLOWCHART:


SUPERMERCATO - FLOWCHART
P1
INIZIO
NUM_SUP
I
NUM_SUP ≤ 0
V
F
i = 0
SUP[i].NOME
SUP[i].COGNOME
SUP[i].INDIRIZZO
SUP[i].NUM DIP
SUP[i].FATT_ULT_ANNO
I
i = i + 1
i < NUM_SUP
V
F
FINE
P2
INIZIO
COGNOME_RESP_RIC
NOME_RESP_RIC
I
FLAG = 0
i = 0
SUP[i].NOME = NOME_RESP_RIC AND SUP[i].COGNOME = COGNOME_RESP_RIC
V
F
SUP[i].NOME
SUP[i].COGNOME
SUP[i].INDIRIZZO
SUP[i].NUM DIP
SUP[i].FATT_ULT_ANNO
O
FLAG = 1
i = i + 1
i < NUM_SUP AND FLAG = 0
V
F
FLAG = 0
V
F
RICERCA FINITA
O
FINE





PROGRAMMA IN C

```
/*****************************************************************************



Supermercato



*****************************************************************************/

#include <stdio.h>

#include <string.h>

#define STRINGA 50

#define MAX_SUP 20



//definire struttura

typedef struct{

    char cognome[STRINGA];

    char nome[STRINGA];

    char citta[STRINGA];

    int num_dip;

    float fattura;

} supermercato_t; //_t per evitare di scrivere ogni volta la struct



//dichiarazione funzioni

int caricamento(supermercato_t sup[]);

void stampa_supermercato(supermercato_t sup[], int num_sup);
```

```
void ricerca_responsabile(supermercato_t sup[], int num_sup);

void conta_supermercati(supermercato_t sup[], int num_sup);

void massimo_dipendenti(supermercato_t sup[], int num_sup);



//main

int main()

{

    supermercato_t super[MAX_SUP];

    int dim_super;

    dim_super=caricamento(super);

    printf("\n");

    stampa_supermercato(super,dim_super);

    printf("\n");

    ricerca_responsabile(super, dim_super);

    printf("\n");

    conta_supermercati(super, dim_super);

    printf("\n");

    massimo_dipendenti(super, dim_super);

    return 0;

}
```

```
//funzione P1

int caricamento(supermercato_t sup[]){

    int num_sup;

    do{

        printf("Inserire numero dei supermercati: ");

        scanf("%d", &num_sup);

    }while(num_sup<0 || num_sup>MAX_SUP || num_sup==0);

    for(int i=0; i<num_sup; i++){

        printf("\nInserire dati ");

        printf("\ncognome del responsabile: ");

        scanf("%s", sup[i].cognome);

        printf("\nNome del responsabile: ");

        scanf("%s", sup[i].nome);

        printf("\nLa città: ");

        scanf("%s", sup[i].citta);

        printf("\nNumero dipendenti: ");

        scanf("%d", &sup[i].num_dip);

        printf("\nFatturato: ");

        scanf("%f", &sup[i].fattura);

    }

    return (num_sup);

}
```

```
//funzione di stampataggio

void stampa_supermercato(supermercato_t sup[], int num_sup){

    for(int i=0; i<num_sup; i++){

        printf("\n I dati: ");

        printf("\ncognome e nome del responsabile: %s %s", sup[i].cognome,
sup[i].nome);

        printf("\nLa città: %s", sup[i].citta);

        printf("\nNumero dipendenti: %d", sup[i].num_dip);

        printf("\nFatturato: %.2f", sup[i].fattura);

    }

}



//funzione di P2

void ricerca_responsabile(supermercato_t sup[], int num_sup){

    char nome_resp_ric[STRINGA];

    char cognome_ric[STRINGA];

    short flag=0;

    printf("\n cognome e nome del responsabile da ricercare: ");

    scanf("%s %s", cognome_ric, nome_resp_ric);

    for(int i=0;(i<num_sup && flag==0);i++){

        if((strcmp(sup[i].cognome, cognome_ric)==0) && (strcmp(sup[i].nome,
```

```
nome_resp_ric)==0)){

             printf("\n I dati: ");

            printf("\ncognome e nome del responsabile: %s %s", sup[i].cognome,
sup[i].nome);

            printf("\nLa città: %s", sup[i].citta);

            printf("\nNumero dipendenti: %d", sup[i].num_dip);

            printf("\nFatturato: %.2f", sup[i].fattura);

            flag=1;

        }

    }

    if(flag==0){

        printf("\n Non esiste un supermercato con questo titolare");

    }

}



//funzione di P3

void conta_supermercati(supermercato_t sup[], int num_sup){

    char citta_ricerca[STRINGA];

    int totale_supermercati=0;

    printf("\nInserire la città da cercare: ");

    scanf("%s", citta_ricerca);

    for(int i=0;i<num_sup;i++){
```

```
        if(strcmp(sup[i].citta, citta_ricerca)==0) totale_supermercati++;

    }

    if(totale_supermercati==0) printf("\nNon ci sono i supermercati in qesta
città ");

    else printf("\nIl numero dei supermercati in quella città sono: %d",
totale_supermercati);

}



//funzione di P4

void massimo_dipendenti(supermercato_t sup[], int num_sup){

    int massimo_idx=0;

    for(int i=1; i<num_sup;i++){

        if(sup[i].num_dip>sup[massimo_idx].num_dip) massimo_idx= i;

    }

    for(int i=0; i<num_sup; i++){

        printf("\n I dati: ");

        printf("\ncognome e nome del responsabile: %s %s",
sup[massimo_idx].cognome, sup[massimo_idx].nome);

        printf("\nLa città: %s", sup[massimo_idx].citta);

        printf("\nNumero dipendenti: %d", sup[massimo_idx].num_dip);

        printf("\nFatturato: %.2f", sup[massimo_idx].fattura);

    }

}
```

LINK PER ANALIZZARE MEGLIO IL CODICE

CODICE CON INSERIMENTO DI 2 NUOVE FUNZIONI

ALGORITMO DI ORDINAMENTO

```
#include <stdio.h>
#include <string.h>

#define STRINGA 50
#define MAX_SUP 20


typedef struct {
        char cognome[STRINGA];
        char nome[STRINGA];
        char via[STRINGA];
        char citta[STRINGA];
        int num_dip;
        float fatt;
} supermercato_t;

int caricamento(supermercato_t sup[]);
void stampa_supermercati(supermercato_t sup[], int num_sup);
void ordina_supermercati (supermercato_t sup[], int num_sup);
void ordina_selection (supermercato_t sup[], int num_sup);

int main()
{
        supermercato_t super[MAX_SUP];
        int dim_super;
        dim_super=caricamento(super);

        ordina_supermercati(super, dim_super);

        stampa_supermercati(super, dim_super);

        ordina_selection (super, dim_super);

        stampa_supermercati(super, dim_super);

        return 0;
```

```
}

int caricamento(supermercato_t sup[]) {
        int num_sup;

        do {
                printf("\n Inserire numero supermercati -->");
                scanf("%d", &num_sup);
        } while (num_sup <= 0 || num_sup > MAX_SUP);

        for(int i=0; i<num_sup; i++) {
                printf("\n Inserire i dati del supermercato");
                printf("\n cognome e nome del responsabile -->");
                scanf("%s %s", sup[i].cognome, sup[i].nome);
                printf("\n indirizzo -->");
                scanf("%s", sup[i].via);
                printf("\n citta -->");
                scanf("%s", sup[i].citta);
                printf("\n numero dipendenti -->");
                scanf("%d", &sup[i].num_dip);
                printf("\n fatturato -->");
                scanf("%f", &sup[i].fatt);

        }
        return (num_sup);

}

void stampa_supermercati(supermercato_t sup[], int num_sup) {
        for(int i=0; i<num_sup; i++) {
                printf("\n\n I dati del supermercato sono: ");
                printf("\n cognome e nome del responsabile --> %s %s",
sup[i].cognome, sup[i].nome);
                printf("\n indirizzo --> %s", sup[i].via);
                printf("\n citta --> %s", sup[i].citta);
                printf("\n numero dipendenti --> %d", sup[i].num_dip);
                printf("\n fatturato --> %f", sup[i].fatt);

        }
}

void ordina_supermercati (supermercato_t sup[], int num_sup) {
    int p =num_sup; //passo dell'ordinamento
```

```
    int scambio;
    supermercato_t temp;

    do {
        scambio=0;
        for (int i = 0; i<(p-1); i++){
            if (sup[i].fatt > sup[i+1].fatt)
            {
                temp=sup[i];
                sup[i]=sup[i+1];
                sup[i+1]=temp;
                scambio = 1;
            }
        }
        p=p-1;

    }while (scambio && p > 0);

}

void ordina_selection (supermercato_t sup[], int num_sup) {

    int idx_min;
    float min;
    supermercato_t temp;

    for (int i=0; i<num_sup-1; i++) {
        min= sup[i].fatt;
        idx_min = i;
        for(int j=i+1; j<num_sup; j++){
            if(sup[j].fatt < min) {
                min=sup[j].fatt;
                idx_min=j;
            }
        }
        temp=sup[i];
        sup[i]=sup[idx_min];
        sup[idx_min]=temp;

    }
}
```

Nella funzione $ordinaSupermercati$ utilizziamo l'algoritmo di ordinamento **BUBBLE SORT**, un algoritmo di ordinamento basato sugli scambi tra le celle successive, vengono confrontati i fatturati della prima cella e della seconda, ordinandoli in ordine crescente.

Nella funzione $ordinaSelection$ utilizziamo l'algoritmo di selezione **SELECTION SORT**, un algoritmo di ordinamento basato sulla ricerca del valore più piccolo nella lista (fattura nel nostro caso) e viene scambiato con la posizione corrente, si esegue questa operazione fin quando tutti gli elementi non sono stati analizzati ed il ciclo si fermerà quanto troverà l'elemento minore della lista.

- Maggiori informazioni per gli ordinamenti
- PER VISUALIZZARE MEGLIO IL CODICE